> 🛠️ **2026.10 수정본 (v11)** — 원본: ch03_05_03_비지도학습_얼굴데이터셋분석_colab.ipynb
>
> 최신 scikit-learn은 `pca.inverse_transform()`에 1차원 배열(중심점 하나)을 넣으면 에러를 내요. `center.reshape(1, -1)`로 2차원으로 바꿔 넣고, 같은 호출을 내부에서 하는 `mglearn.plots.plot_kmeans_faces`에는 2차원으로 바꿔 주는 작은 껍데기(`PCA2D`)를 넘기도록 했습니다. KMeans는 scikit-learn 1.4부터 기본 시도 횟수가 줄어서 `n_init=10`을 적어 예전과 같은 방식으로 돌렸어요. 코랩에서 바로 돌도록 `mglearn` 설치 줄의 주석도 풀었어요.

얼굴 데이터 셋으로 군집 알고리즘 비교

학습 내용

- 얼굴 데이터 셋을 이용한 다양한 비지도 학습 분석을 수행해 봅니다.

목차

01. 라이브러리 불러오기 및 데이터 준비
02. PCA, DBSCAN 활용한 데이터 분석
03. 모델을 활용한 이상치 검출
04. K-Mean로 얼굴 데이터 분석

01. 라이브러리 불러오기 및 데이터 준비

목차로 이동하기

In [1]:
```
%pip install -q mglearn  # 2026.10 수정: 코랩에는 mglearn이 없어 주석 해제
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import matplotlib.pyplot as plt
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.decomposition import NMF
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
```

In [3]:
```
# 데이터 가져오기
from sklearn.datasets import fetch_lfw_people
people = fetch_lfw_people(min_faces_per_person=20, resize=0.7)
```

```
image_shape = people.images[0].shape
print(people.images.shape, image_shape)
```

```
(3023, 87, 65) (87, 65)
```

In [4]:
```
# 이미지 확인
fig, axes = plt.subplots(2, 5, figsize=(15, 8),
                         subplot_kw={'xticks': (), 'yticks': ()})
for target, image, ax in zip(people.target, people.images, axes.ravel()):
    ax.imshow(image)
    ax.set_title(people.target_names[target])
```



In [5]:
```
# 하나의 이미지에 편중되지 않도록, 최대 50장까지만 가져오기
mask = np.zeros(people.target.shape, dtype=np.bool)
for target in np.unique(people.target):
    mask[np.where(people.target == target)[0][:50]] = 1

X_people = people.data[mask]
y_people = people.target[mask]

# 0~255 사이의 흑백 이미지의 픽셀 값을 0~1 사이로 스케일 조정합니다.
# (옮긴이) MinMaxScaler를 적용하는 것과 거의 동일합니다.
X_people = X_people / 255.

# 최대 50장을 뽑는 것을 통해 3023장의 이미지가 2063장의 이미지로 변경.
print(X_people.shape, y_people.shape)
```

```
(2063, 5655) (2063,)
```

02. PCA, DBSCAN 활용한 데이터 분석

목차로 이동하기

PCA 주성분 분석

In [6]:
```
# LFW 데이터에서 고유얼굴을 찾은 다음 데이터를 변환합니다
# 5655 이미지 특징 중에 100개의 주성분을 추출
from sklearn.decomposition import PCA
pca = PCA(n_components=100, whiten=True, random_state=0)
pca.fit_transform(X_people)
X_pca = pca.transform(X_people)
print(X_pca.shape, X_pca[0:2])
```

```
(2063, 100) [[ 0.7149402   1.1231527   0.24520211  0.29232347 -0.25015548
0.42514125
  -0.5108606  -2.799025   -0.8395094   1.812844    1.5454701  -0.77810866
   1.5662161  -1.0815523  -1.0845875  -0.05881449 -0.63381773 -1.1365035
  -0.2483772  -1.181458   -0.16536193 -0.9734577  -1.0278589   0.8978272
  -1.7694998   0.07777227  0.7836601   3.1730072   1.4895153   0.01099296
   0.31534913 -2.0534575   0.37347534 -0.9574999  -0.972637   -0.64596665
   0.83062595  0.09757229  2.045774    1.1238303  -0.29019916 -0.90915656
  -0.04846112 -0.22773619  1.0878568   2.1447952  -0.97436893 -1.5845567
   0.888129   -0.03487894 -0.54594076 -0.25150734  1.6245232   0.51275045
   0.22232834  0.16155505  0.12719874 -0.71308255  1.5482008  -0.39862844
   0.8354038   0.8562803  -2.189587    1.3490683  -1.7779086  -1.1748631
  -2.1178215   0.17919168 -0.78180546  0.3121853  -1.3199117  -2.388809
   1.2940739   0.98880625  1.0116546   1.1350665  -1.9782841   0.05948393
  -0.18120074 -0.2587347  -2.2053428  -0.62488294 -0.63374406  1.5954993
  -1.2404972   0.8450816   0.44025004 -1.6396098   0.8753521   0.04997319
  -1.639878   -0.34927472 -0.23323448  0.01639982 -0.9821371  -1.7122663
  -1.7927197  -0.11184064 -1.0952748  -0.5747433 ]
 [ 0.26801467  1.2788994  -0.22285326 -1.9462801   0.84350383  1.2425047
   0.89747906  0.86737436 -1.7429743   0.0128303  -1.4853255   0.60217416
   0.31807518  0.87333894  0.10414658  1.0106701  -0.0102571  -0.6922986
  -1.083452    0.5716769   1.3514788   0.9424827  -0.03053444  0.94725406
  -0.58264023  0.15310563 -0.05239871  0.27755046  0.0995262  -0.3005752
  -0.05745739 -1.258201    0.39662448 -0.28786728  0.6394572  -1.55255
   1.7647247  -0.25323159  0.9853278  -0.11275336  0.6832321   1.7217813
  -0.47131222 -1.0014235  -0.3759684  -0.2069121  -0.27980867 -0.23699707
   1.7285388   0.9025178   0.40887785 -0.3346557  -0.8604681  -1.5793737
  -0.12476968  1.6659323  -0.54026693  1.5092648  -0.93579936  0.12544608
  -0.49646378  0.7517239  -1.5887011  -1.8098555   2.0793993  -1.0142971
   0.16873829  0.11522195 -0.81151175  0.9222756   0.11860669  1.4337248
  -0.33195952 -0.03435924  0.08438127 -0.9773561   0.39819714 -0.31323257
   0.51978534 -0.9299836  -0.15723228 -0.745588    0.64544296 -1.3765396
  -0.32466805  0.04692357 -0.7966187   0.8169138  -0.08911137  1.0611821
   1.189149    0.727776    0.9731466  -2.06689     0.8122268   0.9118575
   0.37712088  1.3236406   0.08098095  0.05256421]]
```

DBSCAN

In [7]:
```
# 추출된 100개의 특징을 DBSCAN을 적용하여 데이터 분류
# 기본 매개변수로 DBSCAN을 적용합니다
dbscan = DBSCAN()
labels = dbscan.fit_predict(X_pca)
print("고유한 레이블: {}".format(np.unique(labels)))
```

```
고유한 레이블: [-1]
```

- 레이블이 -1이므로 모든 데이터가 DBSCAN에 의해 잡음 포인트로 레이블됨.
  - 해결 : eps값을 크게 하거나, min_samples값을 낮추기

In [8]:
```
dbscan = DBSCAN(min_samples=3)
labels1 = dbscan.fit_predict(X_pca)
print("고유한 레이블: {}".format(np.unique(labels1)))
```

```
고유한 레이블: [-1]
```

- eps 값을 키워보기
- eps 값을 15로 키운 후, 그룹이 2개 생겼다.

In [9]:
```
dbscan = DBSCAN(min_samples=3, eps=15)
labels2 = dbscan.fit_predict(X_pca)
print("고유한 레이블: {}".format(np.unique(labels2)))
```

```
고유한 레이블: [-1  0]
```

- eps와 min_samples의 값에 의해 그룹이 더 많이 생길 수 있다.

In [10]:
```
dbscan = DBSCAN(min_samples=2, eps=15)
labels3 = dbscan.fit_predict(X_pca)
print("고유한 레이블: {}".format(np.unique(labels3)))
```

```
고유한 레이블: [-1  0  1]
```

잡음 포인트와 클러스터 포인트 확인하기

In [11]:
```
# bincount는 음수를 받을 수 없어서 labels3에 1을 더했습니다.
# 고유한 레이블: [-1  0  1]
# 반환값의 첫 번째 원소는 잡음 포인트의 수입니다.
print("클러스터별 포인트 수: {}".format(np.bincount(labels1 + 1)))
print("클러스터별 포인트 수: {}".format(np.bincount(labels2 + 1)))
print("클러스터별 포인트 수: {}".format(np.bincount(labels3 + 1)))
```

```
클러스터별 포인트 수: [2063]
클러스터별 포인트 수: [  37 2026]
클러스터별 포인트 수: [  35 2026    2]
```

03. 모델을 활용한 이상치 검출

목차로 이동하기

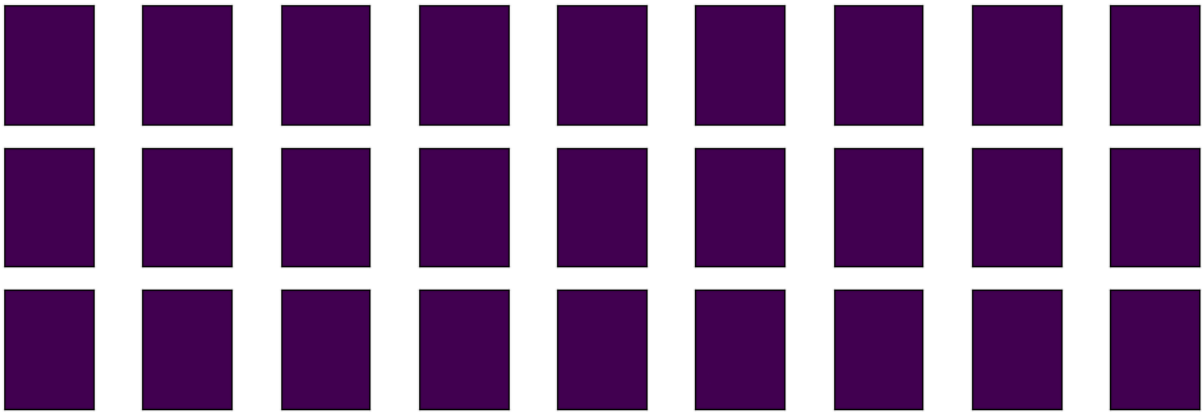
DBSCAN의 알고리즘이 잡음이라 말한 이미지는 확인해 보기

- label1의 -1의 값은 32개, label3의 -1의 값은 30개나 있음.

In [12]:
```
noise = X_people[labels2==-1]  # min_samples=3, eps=15

fig, axes = plt.subplots(3, 9,
                         subplot_kw={'xticks': (), 'yticks': ()},
                         figsize=(12, 4))

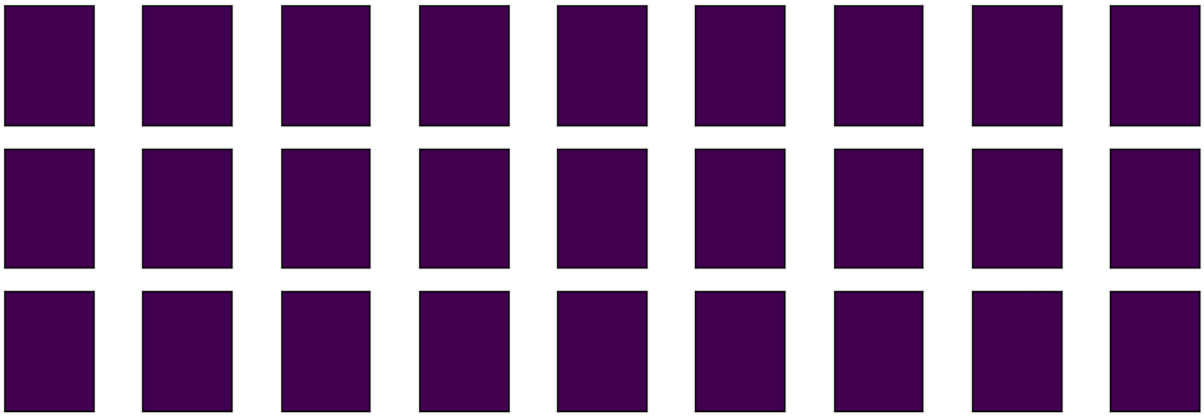
for image, ax in zip(noise, axes.ravel()):
    ax.imshow(image.reshape(image_shape), vmin=0, vmax=1)
```



In [13]:
```
noise = X_people[labels3==-1]  # min_samples=2, eps=15

fig, axes = plt.subplots(3, 9,
                         subplot_kw={'xticks': (), 'yticks': ()},
                         figsize=(12, 4))

for image, ax in zip(noise, axes.ravel()):
    ax.imshow(image.reshape(image_shape), vmin=0, vmax=1)
```



- 이미지 얼굴 앞에 안경, 손 등의 다른 물건이 있음.
- 모자와 얼굴 각도가 조금 다름.

이와 같이 특이한 것을 찾아내는 것을 이상치 검출(outlier detection)이라 한다.

- 실제 애플리케이션이라면 이미지 여백을 잘라 일정한 데이터 셋을 만드는 것이 좋음.

- 큰 클러스터 하나보다 더 많은 클러스터를 찾으려면 eps를 0.5(기본값) ~ 15 사이 정도로 줄여야 함.

In [14]:
```
dbscan = DBSCAN(min_samples=2, eps=10)
labels4 = dbscan.fit_predict(X_pca)
print("고유한 레이블: {}".format(np.unique(labels4)))
```

```
고유한 레이블: [-1  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17]
```

eps에 따른 클러스터 수의 크기와 크기

In [15]:
```
for eps in [1,3,5,7,9,11,13]:
    print("\n eps = ", eps)
    dbscan = DBSCAN(eps =  eps, min_samples=3)
    labels = dbscan.fit_predict(X_pca)
    print("클러스터 수 : ", len(np.unique(labels)))
    print("클러스터 크기 : ", np.bincount(labels + 1))
```

```
 eps =  1
클러스터 수 :  1
클러스터 크기 :  [2063]

 eps =  3
클러스터 수 :  1
클러스터 크기 :  [2063]

 eps =  5
클러스터 수 :  2
클러스터 크기 :  [2059    4]

 eps =  7
클러스터 수 :  8
클러스터 크기 :  [1954   75    4   14    6    4    3    3]

 eps =  9
클러스터 수 :  3
클러스터 크기 :  [1199  861    3]

 eps =  11
클러스터 수 :  2
클러스터 크기 :  [ 403 1660]

 eps =  13
클러스터 수 :  2
클러스터 크기 :  [ 119 1944]
```

- 큰 클러스터가 하나 있고, 그 이외에는 작은 클러스터
  - 확연히 구별되는 얼굴 이미지가 두 세 개가 아닌 모든 이미지는 거의 동일하게 나머지 이미지들과 비슷(또는 비슷하지 않음)것을 의미

작은 클러스터가 많이 만들어진 eps=7, min_samples=3의 결과 확인

In [16]:
```
X_pca.shape
```

Out[16]:
```
(2063, 100)
```

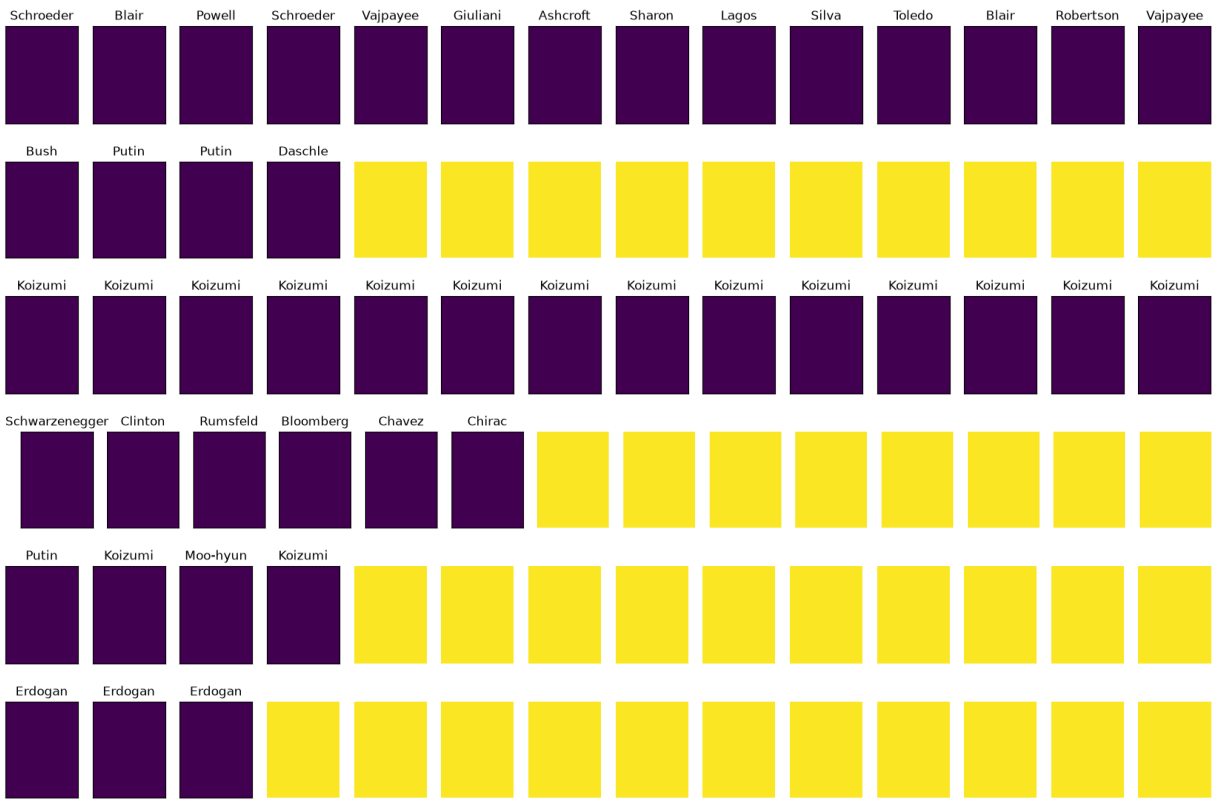
In [17]:
```
dbscan = DBSCAN(eps =  7, min_samples=3)
labels = dbscan.fit_predict(X_pca)
np.unique(labels)
```

Out[17]:
```
array([-1,  0,  1,  2,  3,  4,  5,  6])
```

In [18]:
```
# labels : 만들어진 그룹
# eps =  7, min_samples=3
for cluster in range(max(labels) + 1):
    mask = labels == cluster
    n_images =  np.sum(mask)
    fig, axes = plt.subplots(1, 14, figsize=(14*1.5, 4),
                             subplot_kw={'xticks': (), 'yticks': ()})
    i = 0

    for image, label, ax in zip(X_people[mask], y_people[mask], axes):
        ax.imshow(image.reshape(image_shape), vmin=0, vmax=1)
        ax.set_title(people.target_names[label].split()[-1])
        i += 1

    for j in range(len(axes) - i):
        axes[j+i].imshow(np.array([[1]*65]*87), vmin=0, vmax=1)
        axes[j+i].axis('off')
```





04. k-평균으로 얼굴 데이터 분석

목차로 이동하기

- DBSCAN에서는 하나의 큰 클러스터 외에는 만들 수 없음.
- 병합군집과 K-means은 사용자가 지정하여 비슷한 크기의 클러스터를 만들 수 있음.

In [19]:
```
from sklearn.cluster import KMeans
```

In [20]:
```
# k-평균으로 클러스터를 추출합니다
km = KMeans(n_clusters=10, random_state=0, n_init=10)  # 2026.10 수정: scikit
labels_km = km.fit_predict(X_pca)
print("k-평균의 클러스터 크기: {}".format(np.bincount(labels_km)))
```

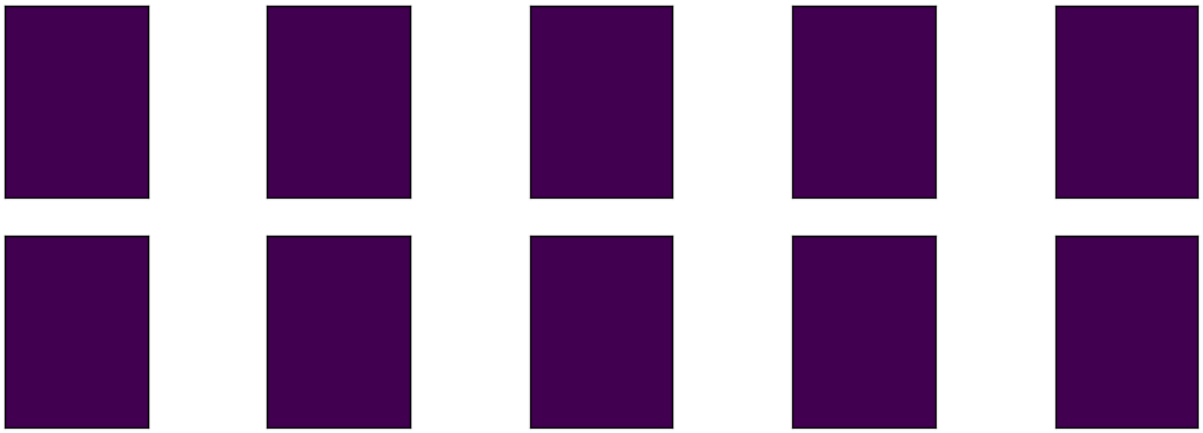
```
k-평균의 클러스터 크기: [122 286 202 220  60 327 363  80 205 198]
```

In [21]:
```
fig, axes = plt.subplots(2, 5,
                         subplot_kw={'xticks': (), 'yticks': ()},
                         figsize=(12, 4))

print(km.cluster_centers_.shape) # 10개 그룹의 중심점.
print(image_shape)

for center, ax in zip(km.cluster_centers_, axes.ravel()):
    print(center.shape, image_shape)
    ax.imshow(pca.inverse_transform(center.reshape(1, -1)).reshape(image_sha
              vmin=0, vmax=1)
```

```
(10, 100)
(87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
(100,) (87, 65)
```



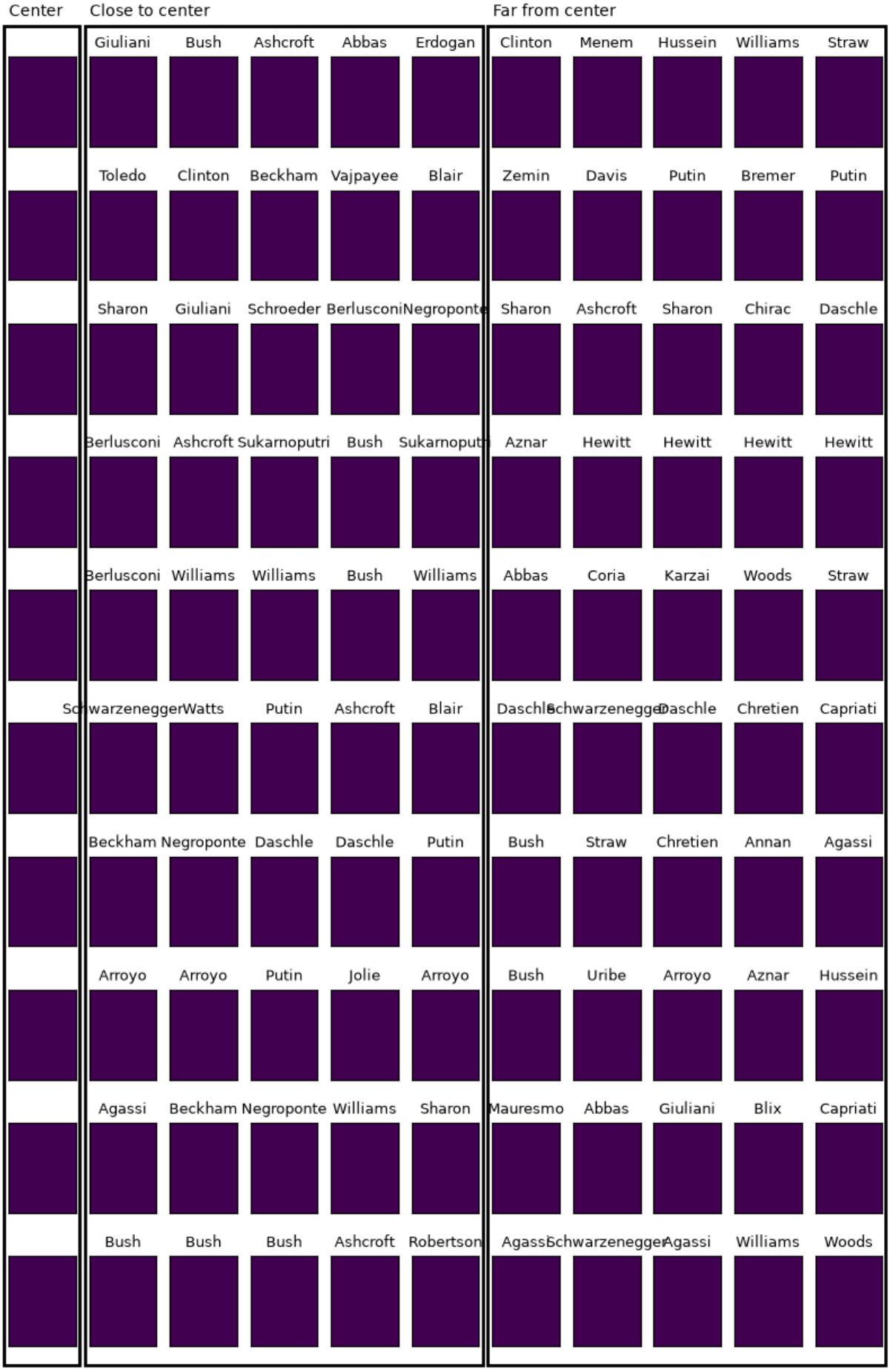
- k-평균이 찾은 클러스터 중심은 매우 부드러운 얼굴 이미지.
- 89개에서 326개까지 얼굴 이미지의 평균이다.
- 차원이 감소된 PCA 성분이 이미지를 더 부드럽게 만든다

각 클러스터에서 가장 대표 이미지(가장 가까운), 가장 동떨어진 이미지(중심에서 가장 먼 이미지) 다섯개

In [22]:
```
import mglearn
```

In [23]:
```
# 2026.10 수정: mglearn 내부가 pca.inverse_transform(1차원 배열)을 호출하는데, 최신 s
class PCA2D:
    def __init__(self, pca): self.pca = pca
    def inverse_transform(self, X): return self.pca.inverse_transform(np.atl

mglearn.plots.plot_kmeans_faces(km, PCA2D(pca), X_pca, X_people,
                                y_people, people.target_names)
```


Center
Close to center
Far from center
Giuliani Bush Ashcroft Abbas Erdogan
Clinton Menem Hussein Williams Straw
Toledo Clinton Beckham Vajpayee Blair
Zemin Davis Putin Bremer Putin
Sharon Giuliani Schroeder Berlusconi Negroponte
Sharon Ashcroft Sharon Chirac Daschle
Berlusconi Ashcroft Sukarnoputri Bush Sukarnoputri
Aznar Hewitt Hewitt Hewitt Hewitt
Berlusconi Williams Williams Bush Williams
Abbas Coria Karzai Woods Straw
Schwarzenegger Watts Putin Ashcroft Blair
Daschle Schwarzenegger Daschle Chretien Capriati
Beckham Negroponte Daschle Daschle Putin
Bush Straw Chretien Annan Agassi
Arroyo Arroyo Putin Jolie Arroyo
Bush Uribe Arroyo Aznar Hussein
Agassi Beckham Negroponte Williams Sharon
Mauresmo Abbas Giuliani Blix Capriati
Bush Bush Bush Ashcroft Robertson
Agassi Schwarzenegger Agassi Williams Woods

- 두번째 클러스터가 웃는 얼굴.
- 나머지 클러스터는 얼굴 각도 중시